

Análisis Estadístico Completo: Correlación AGN HAWC — Neutrinos IceCube

Búsqueda de correlación multimensajero entre 135 AGN del catálogo HAWC y alertas de neutrinos ICECAT-1

Objetivo: Determinar si existe correlación espacial estadísticamente significativa entre 135 AGN detectados por HAWC (seleccionados por procesos leptónicos) y 324 alertas de neutrinos de alta energía del catálogo ICECAT-1 de IceCube.

Hipótesis nula (H_0): Las posiciones de los AGN no están correlacionadas con las posiciones de los neutrinos más allá del azar.

Contexto físico: El catálogo HAWC fue filtrado para fuentes cuya emisión de rayos gamma se atribuye predominantemente a procesos leptónicos (SSC, IC). Las fuentes con evidencia de emisión conjunta de neutrinos (e.g., TXS 0506+056, NGC 1068) son mayoritariamente hadrónicas. Un resultado nulo es *esperado y físicamente consistente*.---

```
In [1]: import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
import os, json, warnings
warnings.filterwarnings('ignore')

# Configuración de figuras
plt.rcParams.update({
    'font.size': 12, 'axes.labelsize': 14, 'axes.titlesize': 14,
    'figure.dpi': 120, 'savefig.dpi': 150, 'savefig.bbox': 'tight'
})

np.random.seed(42)
# voy a usar random seeds de nuevo, puesto que vectorizo cada iteracion, no
# como en el anterior.
N_MC = 10_000 # Iteraciones de Monte Carlo
```

1. Carga y limpieza de datos

```
In [2]: # Cargar datos
DATA_DIR = 'data'

agn_df = pd.read_excel(os.path.join(DATA_DIR, 'AGN_135 -
copiaPRUEBAS.xlsx'))
icecube_df = pd.read_csv(os.path.join(DATA_DIR,
'icecube_copiaPRUEBAS.csv'))

# TXS 0506+056 (agregado como referencia, no pertenece al catálogo
original)
txs_mask = agn_df['source'] == 'J0509.9+0550'
agn_txs = agn_df[txs_mask].copy()
agn_clean = agn_df[~txs_mask].copy().reset_index(drop=True)

print(f"AGN catálogo HAWC: {len(agn_clean)}")
print(f"Alertas IceCube (ICECAT-1): {len(icecube_df)}")
print(f"\nClases AGN:")
print(agn_clean['class'].str.lower().value_counts())
print(f"\nRango Dec AGN: [{agn_clean['Dec_deg'].min():.1f}°,
{agn_clean['Dec_deg'].max():.1f}°]")
print(f"Rango Dec IceCube: [{icecube_df['DEC'].min():.1f}°,
{icecube_df['DEC'].max():.1f}°]")
```

```
AGN catálogo HAWC: 135
Alertas IceCube (ICECAT-1): 324
```

```
Clases AGN:
class
bll      114
bcu       8
rdg       6
fsrq      6
sbg       1
Name: count, dtype: int64
```

```
Rango Dec AGN: [-20.8°, 58.9°]
Rango Dec IceCube: [-86.1°, 84.6°]
```

2. Funciones auxiliares

Distancia angular (Vincenty, esfera) $\cos\theta = \sin\delta_1 \sin\delta_2 + \cos\delta_1 \cos\delta_2 \cos(\alpha_1 - \alpha_2)$

Contorno elíptico (aproximación campo plano) Un AGN en (α, δ) está dentro de la elipse de error del neutrino si: $\left(\frac{\Delta\alpha \cos\delta_\nu}{\sigma_\alpha}\right)^2 + \left(\frac{\Delta\delta}{\sigma_\delta}\right)^2 \leq 1$

```
In [3]: def count_elliptical_coincidences(agn_ra, agn_dec, nu_ra, nu_dec, ra_err,
dec_err):
    """Cuenta AGN dentro de contornos elípticos 90% CL de neutrinos
    (vectorizado)."""
    dra = agn_ra[:, None] - nu_ra[None, :]
    dra = np.where(dra > 180, dra - 360, dra)
    dra = np.where(dra < -180, dra + 360, dra)
    dra_corr = dra * np.cos(np.radians(nu_dec[None, :]))
    ddec = agn_dec[:, None] - nu_dec[None, :]
    dist_norm = (dra_corr / ra_err[None, :])**2 + (ddec / dec_err[None,
:])**2
    has_match = np.any(dist_norm <= 1.0, axis=1)
    return int(np.sum(has_match))

def count_circular_coincidences(src_ra_rad, src_dec_rad, nu_ra_rad,
                                sin_dec_nu, cos_dec_nu, theta_max_rad):
    """Cuenta fuentes con al menos un neutrino dentro de theta_max
    (esfera)."""
    cos_theta = (np.sin(src_dec_rad)[:, None] * sin_dec_nu[None, :] +
                 np.cos(src_dec_rad)[:, None] * cos_dec_nu[None, :] *
                 np.cos(src_ra_rad[:, None] - nu_ra_rad[None, :]))
    cos_theta = np.clip(cos_theta, -1, 1)
    return int(np.sum(np.any(cos_theta >= np.cos(theta_max_rad), axis=1)))

def get_min_angular_distances(src_ra_rad, src_dec_rad, nu_ra_rad,
nu_dec_rad):
    """Distancia angular mínima de cada fuente a su neutrino más
    cercano."""
    cos_theta = (np.sin(src_dec_rad)[:, None] * np.sin(nu_dec_rad)[None, :]
+

```

```

        np.cos(src_dec_rad)[:, None] * np.cos(nu_dec_rad)[None, :])
*
        np.cos(src_ra_rad[:, None] - nu_ra_rad[None, :]))
cos_theta = np.clip(cos_theta, -1, 1)
return np.min(np.arccos(cos_theta), axis=1)

```

```

In [4]: # Preparar arrays
agn_ra = agn_clean['RA_deg'].values
agn_dec = agn_clean['Dec_deg'].values
nu_ra = icecube_df['RA'].values
nu_dec = icecube_df['DEC'].values
ra_err = (icecube_df['RA_ERR_PLUS'].values +
np.abs(icecube_df['RA_ERR_MINUS'].values)) / 2.0
dec_err = (icecube_df['DEC_ERR_PLUS'].values +
np.abs(icecube_df['DEC_ERR_MINUS'].values)) / 2.0
signalness = icecube_df['SIGNAL'].values
energy = icecube_df['ENERGY'].values
agn_ts = agn_clean['TS'].values

# Coordenadas en radianes
src_ra_rad = np.deg2rad(agn_ra)
src_dec_rad = np.deg2rad(agn_dec)
nu_ra_rad = np.deg2rad(nu_ra)
nu_dec_rad = np.deg2rad(nu_dec)
sin_dec_nu = np.sin(nu_dec_rad)
cos_dec_nu = np.cos(nu_dec_rad)

print(f"Arrays preparados: {len(agn_ra)} AGN x {len(nu_ra)} neutrinos")

```

Arrays preparados: 135 AGN x 324 neutrinos

3. Coincidencias con contornos elípticos (90% CL)

Usamos las regiones de error asimétricas reportadas por IceCube para definir contornos elípticos. Aleatorizamos RA uniformemente en $[0^\circ, 360^\circ)$ manteniendo Dec fija (conservando la distribución en declinación del catálogo HAWC).

```

In [5]: # Coincidencias reales
n_real_ellip = count_elliptical_coincidences(agn_ra, agn_dec, nu_ra,
nu_dec, ra_err, dec_err)
print(f"Coincidencias reales (elíptico 90% CL): {n_real_ellip}")

# Monte Carlo: scrambling RA, Dec fija
mc_ellip = np.zeros(N_MC, dtype=int)
for i in range(N_MC):
    ra_scrambled = np.random.uniform(0, 360, size=len(agn_ra))
    mc_ellip[i] = count_elliptical_coincidences(ra_scrambled, agn_dec,
nu_ra, nu_dec, ra_err, dec_err)

mean_e, std_e = np.mean(mc_ellip), np.std(mc_ellip)
p_ellip = np.mean(mc_ellip >= n_real_ellip)
z_ellip = (n_real_ellip - mean_e) / std_e

print(f"\nMonte Carlo ({N_MC:,} iter):")
print(f"  Media: {mean_e:.1f} ± {std_e:.1f}")
print(f"  Rango: [{mc_ellip.min()}, {mc_ellip.max()}]")
print(f"  p-value: {p_ellip:.4f}")
print(f"  z = {z_ellip:.2f}σ")
print(f"\n→ RESULTADO: Compatible con fondo aleatorio (H0 no rechazada)")

```

Coincidencias reales (elíptico 90% CL): 32

Monte Carlo (10,000 iter):

Media: 34.9 ± 4.9

Rango: [17, 53]

p-value: 0.7527

z = -0.60σ

→ RESULTADO: Compatible con fondo aleatorio (H₀ no rechazada)

```

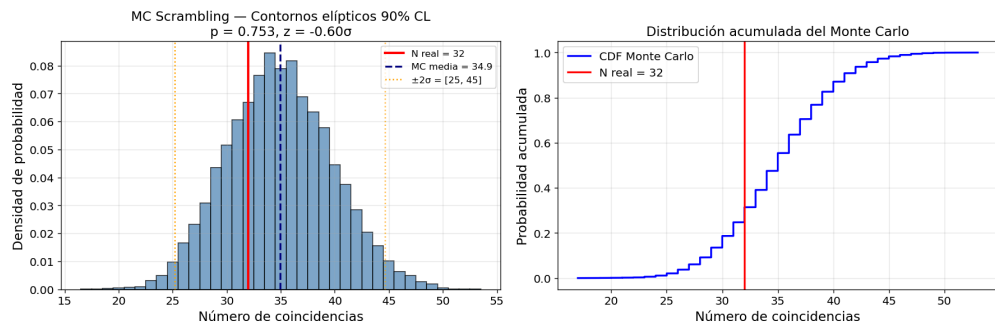
In [6]: fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Histograma
ax1 = axes[0]
bins = np.arange(mc_ellip.min() - 0.5, mc_ellip.max() + 1.5, 1)
ax1.hist(mc_ellip, bins=bins, edgecolor='black', alpha=0.7,
color='steelblue', density=True)
ax1.axvline(n_real_ellip, color='red', linewidth=2.5, label=f'N real = {n_real_ellip}')
ax1.axvline(mean_e, color='navy', linewidth=2, linestyle='--', label=f'MC media = {mean_e:.1f}')
ax1.axvline(mean_e + 2*std_e, color='orange', linewidth=1.5, linestyle=':', alpha=0.7)
ax1.axvline(mean_e - 2*std_e, color='orange', linewidth=1.5, linestyle=':',
label=f' $\pm 2\sigma = [{mean_e - 2*std_e:.0f}, {mean_e + 2*std_e:.0f}]$ ')
ax1.set_xlabel('Número de coincidencias')
ax1.set_ylabel('Densidad de probabilidad')
ax1.set_title(f'MC Scrambling – Contornos elípticos 90% CL\np = {p_ellip:.3f}, z = {z_ellip:.2f} $\sigma$ ')
ax1.legend(fontsize=10)
ax1.grid(True, alpha=0.3)

# CDF
ax2 = axes[1]
sorted_mc = np.sort(mc_ellip)
cdf = np.arange(1, len(sorted_mc) + 1) / len(sorted_mc)
ax2.plot(sorted_mc, cdf, 'b-', linewidth=2, label='CDF Monte Carlo')
ax2.axvline(n_real_ellip, color='red', linewidth=2, label=f'N real = {n_real_ellip}')
ax2.set_xlabel('Número de coincidencias')
ax2.set_ylabel('Probabilidad acumulada')
ax2.set_title('Distribución acumulada del Monte Carlo')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



4. Coincidencias con radio circular fijo

Evaluamos múltiples radios de búsqueda: $\theta \in \{0.5^\circ, 1.0^\circ, 1.5^\circ, 2.0^\circ, 3.0^\circ\}$.

Dos modelos nulos:

1. **Isotrópico**: RA uniforme $[0, 2\pi)$, Dec uniforme en $\sin(\delta)$ → distribución uniforme en la esfera
2. **Banda HAWC**: RA uniforme $[0, 2\pi)$, Dec fija del catálogo → conserva la cobertura de HAWCEl modelo nulo correcto para esta búsqueda es el de **banda HAWC**, ya que los AGN están restringidos al campo de visión del detector.

```

In [7]: theta_grid_deg = np.array([0.5, 1.0, 1.5, 2.0, 3.0])
theta_grid_rad = np.deg2rad(theta_grid_deg)
N_theta = len(theta_grid_deg)

# Coincidencias reales
n_real_theta = np.array([
    count_circular_coincidences(src_ra_rad, src_dec_rad, nu_ra_rad,
                                sin_dec_nu, cos_dec_nu, th)
    for th in theta_grid_rad
])

# ===== MC Isotrópico =====
mc_iso = np.zeros((N_MC, N_theta), dtype=int)
for i in range(N_MC):
    ra_rand = np.random.uniform(0, 2*np.pi, len(agn_ra))
    u = np.random.uniform(-1, 1, len(agn_ra))
    dec_rand = np.arcsin(u)
    sin_d = np.sin(dec_rand)
    cos_d = np.cos(dec_rand)
    for j, th in enumerate(theta_grid_rad):
        cos_th = (sin_d[:, None] * sin_dec_nu[None, :] +
                  cos_d[:, None] * cos_dec_nu[None, :] *
                  np.cos(ra_rand[:, None] - nu_ra_rad[None, :]))
        mc_iso[i, j] = int(np.sum(np.any(np.clip(cos_th, -1, 1) >=
np.cos(th), axis=1)))

# ===== MC Banda HAWC (Dec fija) =====
mc_hawc = np.zeros((N_MC, N_theta), dtype=int)
for i in range(N_MC):
    ra_rand = np.random.uniform(0, 2*np.pi, len(agn_ra))
    sin_d = np.sin(src_dec_rad)
    cos_d = np.cos(src_dec_rad)
    for j, th in enumerate(theta_grid_rad):
        cos_th = (sin_d[:, None] * sin_dec_nu[None, :] +
                  cos_d[:, None] * cos_dec_nu[None, :] *
                  np.cos(ra_rand[:, None] - nu_ra_rad[None, :]))
        mc_hawc[i, j] = int(np.sum(np.any(np.clip(cos_th, -1, 1) >=
np.cos(th), axis=1)))

print("Cálculo completado.")

```

Cálculo completado.


```
In [8]: # Tabla comparativa
print(f"{'':>8} | {'':>6} | {'MC Isotrópico':^25} | {'MC Banda HAWC':^25}")
print(f"{'θ (°)':>8} | {'N_obs':>6} | {'media':>7} {'std':>5} {'p':>8} |
{'media':>7} {'std':>5} {'p':>8}")
print("-" * 75)

p_iso = np.zeros(N_theta)
p_hawc = np.zeros(N_theta)

for j in range(N_theta):
    m_i, s_i = np.mean(mc_iso[:, j]), np.std(mc_iso[:, j])
    m_h, s_h = np.mean(mc_hawc[:, j]), np.std(mc_hawc[:, j])
    p_iso[j] = np.mean(mc_iso[:, j] >= n_real_theta[j])
    p_hawc[j] = np.mean(mc_hawc[:, j] >= n_real_theta[j])
    print(f"{theta_grid_deg[j]:>8.1f} | {n_real_theta[j]:>6} | {m_i:>7.1f}
{s_i:>5.1f} {p_iso[j]:>8.4f} | {m_h:>7.1f} {s_h:>5.1f} {p_hawc[j]:>8.4f}")

print()
print("NOTA IMPORTANTE: Los p-values isotrópicos muestran excesos aparentes
a radios")
print("grandes porque distribuir AGN uniformemente en toda la esfera reduce
la densidad")
print("en la banda de declinación de HAWC ( $\delta \approx -20^\circ$  a  $+60^\circ$ ), donde se
concentran los AGN")
print("y donde IceCube tiene buena sensibilidad. Este es un efecto
GEOMÉTRICO, no físico.")
print("\nEl modelo nulo correcto es el de BANDA HAWC → p-values todos > 0.4
→ SIN SEÑAL.")
```

θ (°)	N_obs	MC Isotrópico				MC Banda HAWC		
		media	std	p		media	std	p

0.5	1	0.8	0.9	0.5543		1.4	1.2	0.7515
1.0	5	3.2	1.8	0.2170		5.5	2.3	0.6604
1.5	11	7.0	2.6	0.0930		11.7	3.2	0.6392
2.0	20	11.8	3.3	0.0134		19.5	3.9	0.4926
3.0	39	23.6	4.3	0.0007		38.2	4.8	0.4723

NOTA IMPORTANTE: Los p-values isotrópicos muestran excesos aparentes a radios grandes porque distribuir AGN uniformemente en toda la esfera reduce la densidad en la banda de declinación de HAWC ($\delta \approx -20^\circ$ a $+60^\circ$), donde se concentran los AGN y donde IceCube tiene buena sensibilidad. Este es un efecto GEOMÉTRICO, no físico.

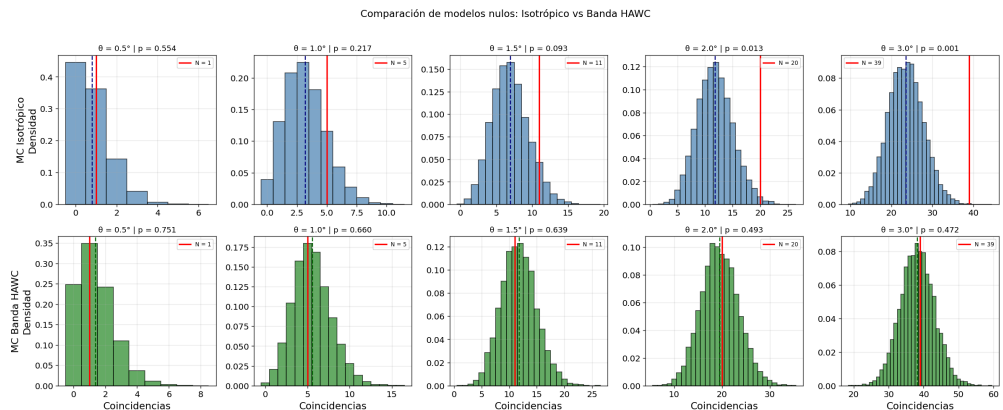
El modelo nulo correcto es el de BANDA HAWC → p-values todos > 0.4 → SIN SEÑAL.

```
In [9]: fig, axes = plt.subplots(2, len(theta_grid_deg), figsize=
(4*len(theta_grid_deg), 8))

for j, th in enumerate(theta_grid_deg):
    # Isotrópico (arriba)
    ax = axes[0, j]
    bins = np.arange(mc_iso[:, j].min()-0.5, mc_iso[:, j].max()+1.5, 1)
    ax.hist(mc_iso[:, j], bins=bins, edgecolor='black', alpha=0.7,
color='steelblue', density=True)
    ax.axvline(n_real_theta[j], color='red', linewidth=2, label=f'N =
{n_real_theta[j]}')
    ax.axvline(np.mean(mc_iso[:, j]), color='navy', linewidth=1.5,
linestyle='--')
    ax.set_title(f' $\theta = \{th\}^\circ \mid p = \{p\_iso[j]:.3f\}$ ', fontsize=11)
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)
    if j == 0: ax.set_ylabel('MC Isotrópico\nDensidad')

    # Banda HAWC (abajo)
    ax = axes[1, j]
    bins = np.arange(mc_hawc[:, j].min()-0.5, mc_hawc[:, j].max()+1.5, 1)
    ax.hist(mc_hawc[:, j], bins=bins, edgecolor='black', alpha=0.7,
color='forestgreen', density=True)
    ax.axvline(n_real_theta[j], color='red', linewidth=2, label=f'N =
{n_real_theta[j]}')
    ax.axvline(np.mean(mc_hawc[:, j]), color='darkgreen', linewidth=1.5,
linestyle='--')
    ax.set_title(f' $\theta = \{th\}^\circ \mid p = \{p\_hawc[j]:.3f\}$ ', fontsize=11)
    ax.set_xlabel('Coincidencias')
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)
    if j == 0: ax.set_ylabel('MC Banda HAWC\nDensidad')

plt.suptitle('Comparación de modelos nulos: Isotrópico vs Banda HAWC',
fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```



5. Look-Elsewhere Effect (corrección por trials)

Al evaluar múltiples radios de búsqueda, aumenta la probabilidad de encontrar una fluctuación por azar. Aplicamos la corrección por trials para obtener el p-value global.

Método: Para cada iteración MC, calculamos el p-value mínimo entre todos los θ . La distribución de estos mínimos nos da el p-value global.

Corrección de Bonferroni (conservadora): $p_{\text{global}} \leq N_{\text{trials}} \times p_{\text{min}}$

Esta para tratar de corregir los errores de tipo I, asegura que la probabilidad de obtener resultados significativos por puro azar sea baja.

```
In [10]: # Usando el MC de banda HAWC (modelo nulo correcto)
min_p_obs = np.min(p_hawc)
best_theta_idx = np.argmin(p_hawc)
print(f"Mínimo p-value (pre-trials, banda HAWC): {min_p_obs:.4f} en  $\theta =$ 
{theta_grid_deg[best_theta_idx]:.1f}°")

# Corrección por trials (empírica)
mc_min_p = np.zeros(N_MC)
for i in range(N_MC):
    p_this = np.array([np.mean(mc_hawc[:, j]) >= mc_hawc[i, j]) for j in
range(N_theta)])
    mc_min_p[i] = np.min(p_this)

p_global_hawc = np.mean(mc_min_p <= min_p_obs)

# Bonferroni
p_bonferroni = min(1.0, min_p_obs * N_theta)

print(f"p-value global (post-trials): {p_global_hawc:.4f}")
print(f"p-value Bonferroni: {p_bonferroni:.4f}")
print(f"\n→ RESULTADO: Todos los tests son compatibles con fondo bajo el
modelo nulo de banda HAWC.")
```

```
Mínimo p-value (pre-trials, banda HAWC): 0.4723 en  $\theta = 3.0^\circ$ 
p-value global (post-trials): 0.7673
p-value Bonferroni: 1.0000
```

→ RESULTADO: Todos los tests son compatibles con fondo bajo el modelo nulo de banda HAWC.

6. Test de Kolmogorov-Smirnov

Comparamos la distribución de distancias angulares mínimas (de cada AGN a su neutrino más cercano) entre los datos reales y el modelo nulo.

```
In [11]: # Distancias mínimas reales
real_min_dists = np.rad2deg(get_min_angular_distances(src_ra_rad,
src_dec_rad, nu_ra_rad, nu_dec_rad))

# Distancias mínimas MC (banda HAWC, 1000 realizaciones)
mc_min_dists_all = []
for i in range(1000):
    ra_rand = np.random.uniform(0, 2*np.pi, len(agn_ra))
    dists = np.rad2deg(get_min_angular_distances(ra_rand, src_dec_rad,
nu_ra_rad, nu_dec_rad))
    mc_min_dists_all.extend(dists)
mc_min_dists_all = np.array(mc_min_dists_all)

# KS test manual
def ks_2sample(d1, d2):
    d1, d2 = np.sort(d1), np.sort(d2)
    all_d = np.sort(np.concatenate([d1, d2]))
    cdf1 = np.searchsorted(d1, all_d, side='right') / len(d1)
    cdf2 = np.searchsorted(d2, all_d, side='right') / len(d2)
    D = np.max(np.abs(cdf1 - cdf2))
    ne = len(d1) * len(d2) / (len(d1) + len(d2))
    z = (np.sqrt(ne) + 0.12 + 0.11/np.sqrt(ne)) * D
    p = max(0, sum(2*(-1)**(k-1)*np.exp(-2*k**2*z**2) for k in range(1,
101)))
    return D, min(p, 1.0)

D_ks, p_ks = ks_2sample(real_min_dists, mc_min_dists_all)
print(f"KS statistic D = {D_ks:.4f}")
print(f"KS p-value = {p_ks:.4f}")
print(f"\nMedia dist. real: {np.mean(real_min_dists):.2f}°")
print(f"Media dist. MC: {np.mean(mc_min_dists_all):.2f}°")
print(f"Mediana real: {np.median(real_min_dists):.2f}°")
print(f"Mediana MC: {np.median(mc_min_dists_all):.2f}°")
```

```
KS statistic D = 0.0720
KS p-value = 0.4725
```

```
Media dist. real: 5.23°
Media dist. MC: 5.45°
Mediana real: 4.51°
Mediana MC: 4.56°
```

```

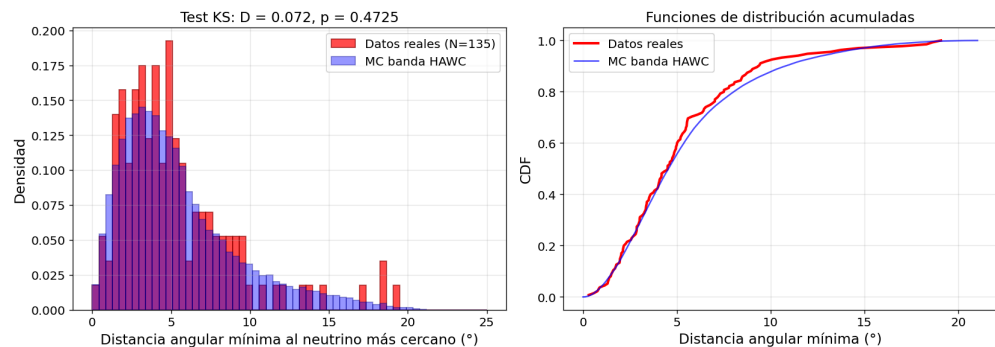
In [12]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

ax1 = axes[0]
bins_dist = np.linspace(0, 25, 60)
ax1.hist(real_min_dists, bins=bins_dist, alpha=0.7, color='red',
density=True,
label=f'Datos reales (N={len(real_min_dists)})',
edgecolor='darkred')
ax1.hist(mc_min_dists_all, bins=bins_dist, alpha=0.4, color='blue',
density=True,
label=f'MC banda HAWC', edgecolor='darkblue')
ax1.set_xlabel('Distancia angular mínima al neutrino más cercano (°)')
ax1.set_ylabel('Densidad')
ax1.set_title(f'Test KS: D = {D_ks:.3f}, p = {p_ks:.4f}')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2 = axes[1]
rs = np.sort(real_min_dists)
ms = np.sort(mc_min_dists_all)
ax2.plot(rs, np.arange(1, len(rs)+1)/len(rs), 'r-', linewidth=2.5,
label='Datos reales')
ax2.plot(ms, np.arange(1, len(ms)+1)/len(ms), 'b-', linewidth=1.5,
alpha=0.7, label='MC banda HAWC')
ax2.set_xlabel('Distancia angular mínima (°)')
ax2.set_ylabel('CDF')
ax2.set_title('Funciones de distribución acumuladas')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



7. Tests ponderados

7a. Ponderado por signalness

$$\text{Score} = \sum_i \max_j (S_j \cdot \mathbb{1}[\theta_{ij} < \theta_{\max}])$$

donde S_j es la probabilidad de que el neutrino j sea de origen astrofísico.

7b. Ponderado por energía

$$\text{Score} = \sum_i \max_j (\log_{10} E_j \cdot \mathbb{1}[\theta_{ij} < \theta_{\max}])$$

```

In [13]: def weighted_score(agn_ra_r, agn_dec_r, nu_ra_r, nu_dec_r, weights,
theta_max_r):
    cos_theta = (np.sin(agn_dec_r)[: , None] * np.sin(nu_dec_r)[None, :] +
                  np.cos(agn_dec_r)[: , None] * np.cos(nu_dec_r)[None, :] *
                  np.cos(agn_ra_r[: , None] - nu_ra_r[None, :]))
    cos_theta = np.clip(cos_theta, -1, 1)
    inside = cos_theta >= np.cos(theta_max_r)
    return np.sum(np.max(inside.astype(float) * weights[None, :], axis=1))

theta_w = np.deg2rad(1.5)

# Signalness
score_S = weighted_score(src_ra_rad, src_dec_rad, nu_ra_rad, nu_dec_rad,
signalness, theta_w)
mc_S = np.array([weighted_score(np.random.uniform(0, 2*np.pi, len(agn_ra)),
src_dec_rad,
                                nu_ra_rad, nu_dec_rad, signalness,
theta_w) for _ in range(N_MC)])
p_S = np.mean(mc_S >= score_S)
z_S = (score_S - np.mean(mc_S)) / np.std(mc_S) if np.std(mc_S) > 0 else 0

# Energía
log_E = np.log10(energy)
score_E = weighted_score(src_ra_rad, src_dec_rad, nu_ra_rad, nu_dec_rad,
log_E, theta_w)
mc_E = np.array([weighted_score(np.random.uniform(0, 2*np.pi, len(agn_ra)),
src_dec_rad,
                                nu_ra_rad, nu_dec_rad, log_E, theta_w) for
_ in range(N_MC)])
p_E = np.mean(mc_E >= score_E)
z_E = (score_E - np.mean(mc_E)) / np.std(mc_E) if np.std(mc_E) > 0 else 0

print(f"Ponderado por signalness ( $\theta=1.5^\circ$ ):")
print(f" Score real: {score_S:.3f}, MC: {np.mean(mc_S):.3f}  $\pm$ 
{np.std(mc_S):.3f}")
print(f" p = {p_S:.4f}, z = {z_S:.2f} $\sigma$ ")
print(f"\nPonderado por  $\log_{10}(E)$  ( $\theta=1.5^\circ$ ):")
print(f" Score real: {score_E:.3f}, MC: {np.mean(mc_E):.3f}  $\pm$ 
{np.std(mc_E):.3f}")
print(f" p = {p_E:.4f}, z = {z_E:.2f} $\sigma$ ")

```

```

Ponderado por signalness ( $\theta=1.5^\circ$ ):
Score real: 5.120, MC: 5.485  $\pm$  1.603
p = 0.5674, z = -0.23 $\sigma$ 

```

```

Ponderado por  $\log_{10}(E)$  ( $\theta=1.5^\circ$ ):
Score real: 22.933, MC: 26.294  $\pm$  7.198
p = 0.6728, z = -0.47 $\sigma$ 

```

```

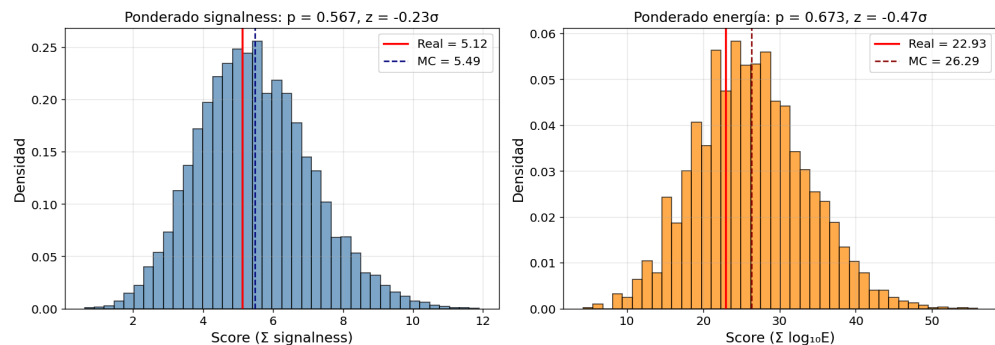
In [14]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

ax1 = axes[0]
ax1.hist(mc_S, bins=40, alpha=0.7, color='steelblue', edgecolor='black',
density=True)
ax1.axvline(score_S, color='red', linewidth=2, label=f'Real =
{score_S:.2f}')
ax1.axvline(np.mean(mc_S), color='navy', linewidth=1.5, linestyle='--',
label=f'MC = {np.mean(mc_S):.2f}')
ax1.set_xlabel('Score ( $\Sigma$  signalness)')
ax1.set_ylabel('Densidad')
ax1.set_title(f'Ponderado signalness: p = {p_S:.3f}, z = {z_S:.2f} $\sigma$ ')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2 = axes[1]
ax2.hist(mc_E, bins=40, alpha=0.7, color='darkorange', edgecolor='black',
density=True)
ax2.axvline(score_E, color='red', linewidth=2, label=f'Real =
{score_E:.2f}')
ax2.axvline(np.mean(mc_E), color='darkred', linewidth=1.5, linestyle='--',
label=f'MC = {np.mean(mc_E):.2f}')
ax2.set_xlabel('Score ( $\Sigma \log_{10} E$ )')
ax2.set_ylabel('Densidad')
ax2.set_title(f'Ponderado energía: p = {p_E:.3f}, z = {z_E:.2f} $\sigma$ ')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



8. Correlación TS HAWC vs número de neutrinos cercanos

Si las fuentes HAWC más significativas (mayor TS) también produjeran neutrinos, esperaríamos una correlación positiva.


```

In [15]: # Contar neutrinos por AGN
individual_counts = np.zeros(len(agn_ra), dtype=int)
for k in range(len(agn_ra)):
    cos_th = (np.sin(src_dec_rad[k]) * sin_dec_nu +
              np.cos(src_dec_rad[k]) * cos_dec_nu *
              np.cos(src_ra_rad[k] - nu_ra_rad))
    individual_counts[k] = np.sum(cos_th >= np.cos(np.deg2rad(1.5)))

# Spearman (manual)
rank_ts = np.argsort(np.argsort(agn_ts)).astype(float)
rank_n = np.argsort(np.argsort(individual_counts)).astype(float)
d = rank_ts - rank_n
n = len(agn_ts)
rs = 1 - 6 * np.sum(d**2) / (n * (n**2 - 1))

print(f"Spearman  $\rho$  = {rs:.4f}")
print(f"Interpretación: {'Correlación negativa' if rs < 0 else 'Correlación positiva'}")
print(f"\nNOTA: La correlación negativa indica que los AGN con mayor TS HAWC")
print(f"no tienden a tener más neutrinos cercanos. Esto es consistente con")
print(f"que la emisión gamma (leptónica) NO implica producción de neutrinos.")

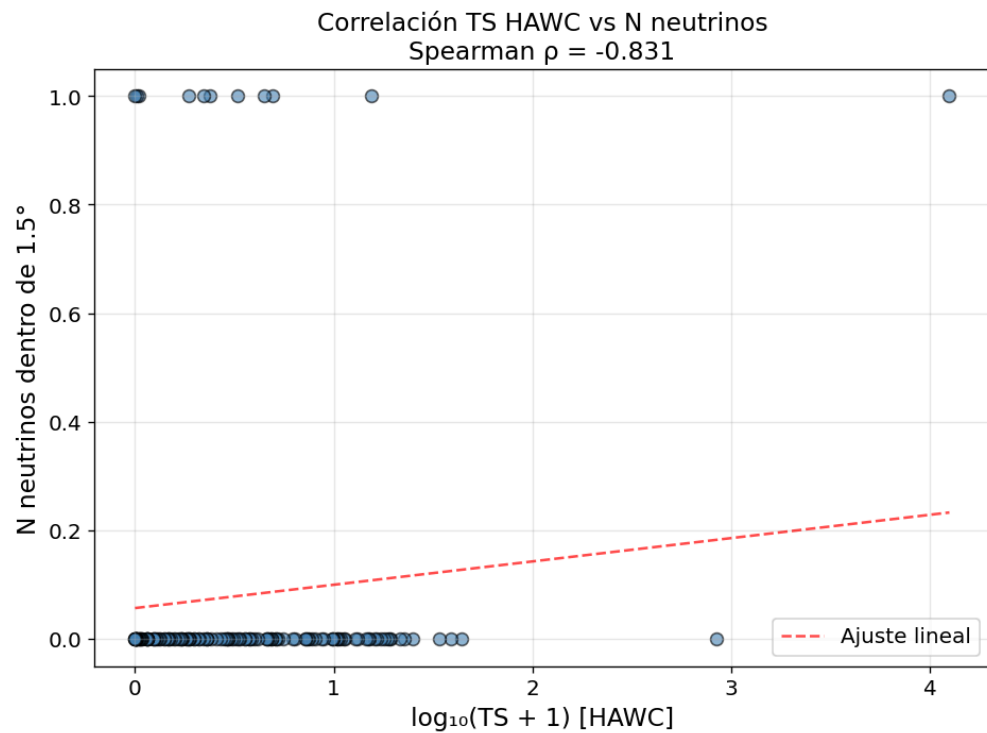
```

Spearman ρ = -0.8312

Interpretación: Correlación negativa

NOTA: La correlación negativa indica que los AGN con mayor TS HAWC no tienden a tener más neutrinos cercanos. Esto es consistente con que la emisión gamma (leptónica) NO implica producción de neutrinos.

```
In [16]: fig, ax = plt.subplots(figsize=(8, 6))
ts_log = np.log10(agn_ts + 1)
ax.scatter(ts_log, individual_counts, c='steelblue', s=50, alpha=0.6,
edgecolors='black')
ax.set_xlabel('log10(TS + 1) [HAWC]')
ax.set_ylabel('N neutrinos dentro de 1.5°')
ax.set_title(f'Correlación TS HAWC vs N neutrinos\nSpearman ρ = {rs:.3f}')
ax.grid(True, alpha=0.3)
z_fit = np.polyfit(ts_log, individual_counts, 1)
x_fit = np.linspace(ts_log.min(), ts_log.max(), 100)
ax.plot(x_fit, np.polyval(z_fit, x_fit), 'r--', linewidth=1.5, alpha=0.7,
label='Ajuste lineal')
ax.legend()
plt.tight_layout()
plt.show()
```



9. Upper limits al número de fuentes multimensajero

Dado que no se detecta señal significativa, calculamos límites superiores a la fracción de AGN leptónicos que podrían tener emisión de neutrinos detectable.

Método: Aproximación Gaussiana del perfil de likelihood: $\text{UL}(95\% \text{ CL}) = \max(0, N_{\text{obs}} - \mu_{\text{MC}}) + 1.645 \cdot \sigma_{\text{MC}}$

```
In [17]: print(f"{'Método':>20} | {'N_obs':>5} | {'Fondo':>8} | {'UL 90%':>7} | {'UL
95%':>7} | {'Frac 95%':>8}")
print("-" * 75)

# Elíptico
excess = max(0, n_real_ellip - mean_e)
ul90_e = excess + 1.282 * std_e
ul95_e = excess + 1.645 * std_e
print(f"{'Elíptico 90% CL':>20} | {n_real_ellip:>5} | {mean_e:>8.1f} |
{ul90_e:>7.1f} | {ul95_e:>7.1f} | {ul95_e/len(agn_ra)*100:>7.1f}%")

# Circulares (banda HAWC)
for j in range(N_theta):
    m = np.mean(mc_hawc[:, j])
    s = np.std(mc_hawc[:, j])
    exc = max(0, n_real_theta[j] - m)
    ul90 = exc + 1.282 * s
    ul95 = exc + 1.645 * s
    print(f"{'θ=' + str(theta_grid_deg[j]) + '° (HAWC)':>20} |
{n_real_theta[j]:>5} | {m:>8.1f} | {ul90:>7.1f} | {ul95:>7.1f} |
{ul95/len(agn_ra)*100:>7.1f}%")

print(f"\n→ Con 95% CL: menos del {ul95_e/len(agn_ra)*100:.0f}% de los
{len(agn_ra)} AGN leptónicos")
print(f"    tienen emisión de neutrinos detectable (método elíptico).")
```

	Método	N_obs	Fondo	UL 90%	UL 95%	Frac
95%	-----					

	Elíptico 90% CL	32	34.9	6.2	8.0	
5.9%						
	θ=0.5° (HAWC)	1	1.4	1.5	1.9	
1.4%						
	θ=1.0° (HAWC)	5	5.5	2.9	3.7	
2.8%						
	θ=1.5° (HAWC)	11	11.7	4.1	5.2	
3.9%						
	θ=2.0° (HAWC)	20	19.5	5.5	6.9	
5.1%						
	θ=3.0° (HAWC)	39	38.2	7.0	8.7	
6.5%						

→ Con 95% CL: menos del 6% de los 135 AGN leptónicos
 tienen emisión de neutrinos detectable (método elíptico).

10. Tabla resumen por fuente

Las 15 fuentes con la distancia angular más pequeña a un neutrino IceCube.

```

In [18]: # Construir tabla
results = []
for k in range(len(agn_ra)):
    dists = np.rad2deg(get_min_angular_distances(
        src_ra_rad[k:k+1], src_dec_rad[k:k+1], nu_ra_rad, nu_dec_rad))
    min_d = dists[0]
    closest = np.argmin(np.arccos(np.clip(
        np.sin(src_dec_rad[k])*np.sin(nu_dec_rad) +
        np.cos(src_dec_rad[k])*np.cos(nu_dec_rad)*np.cos(src_ra_rad[k]-
nu_ra_rad), -1, 1)))

    results.append({
        'AGN': agn_clean['source'].iloc[k],
        'Clase': agn_clean['class'].iloc[k],
        'TS': agn_ts[k],
        'Dist_min': min_d,
        'Neutrino': icecube_df['NAME'].iloc[closest],
        'Signalness': signalness[closest],
        'E_TeV': energy[closest],
        'N_1.5deg': individual_counts[k]
    })

df_res =
pd.DataFrame(results).sort_values('Dist_min').reset_index(drop=True)
print(df_res.head(15).to_string(index=False))

# Guardar
df_res.to_csv('tabla_resumen_completa.csv', index=False)
print(f"\nTabla completa guardada (135 fuentes)")

```

	AGN	Clase	TS	Dist_min	Neutrino	Signalness	E_TeV
N_1.5deg							
J0816.9+2050	b11	3.91	0.308932	IC230217A	0.454	55	
1							
J0211.2+1051	BLL	0.02	0.582079	IC131014A	0.665	293	
1							
J1104.4+3812	BLL	12584.78	0.770367	IC111208A	0.446	123	
1							
J1449.5+2745	b11	1.21	0.829710	IC141114A	0.380	110	
1							
J2322.6+3436	b11	2.30	0.886392	IC221223A	0.795	353	
1							
J0217.1+0836	b11	14.48	1.145242	IC230727A	0.294	113	
1							
J2252.0+4031	b11	3.46	1.309207	IC131108A	0.502	153	
1							
J1402.6+1559	b11	1.38	1.335216	IC221104A	0.316	109	
1							
J1442.8+1200	b11	0.87	1.369677	IC190413A	0.292	107	
1							
J1117.0+2014	b11	0.04	1.397265	IC130408A	0.525	65	
1							
J1603.8+1103	b11	0.00	1.488972	IC160427A	0.451	85	

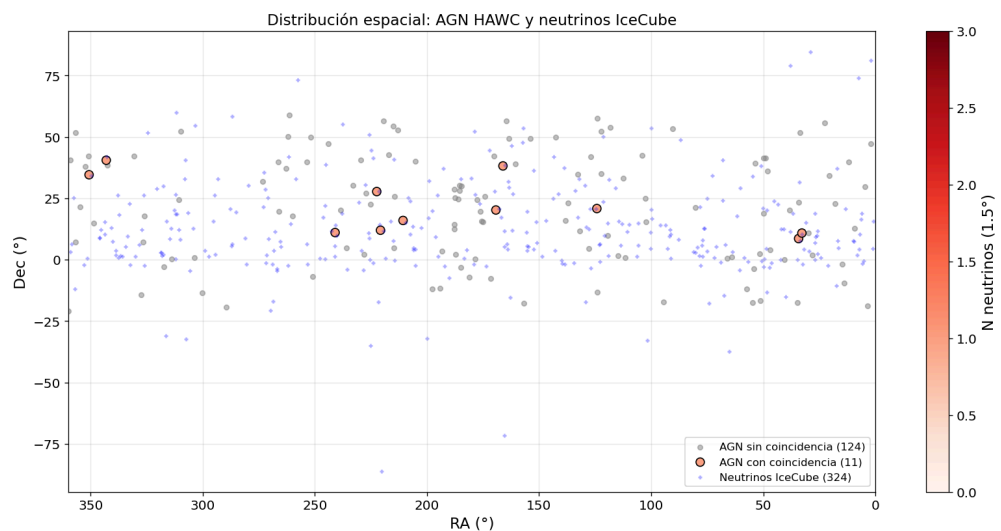
1							
J1554.2+2010	b11	13.88	1.512660	IC120301A	0.825	433	
0							
J0757.1+0957	b11	0.24	1.564616	IC230405A	0.299	110	
0							
J2055.0+0014	b11	0.82	1.611362	IC180528A	0.278	110	
0							
J0308.4+0408	rdg	6.20	1.710746	IC220304A	0.631	263	
0							

Tabla completa guardada (135 fuentes)

11. Mapa del cielo

```
In [21]: fig, ax = plt.subplots(figsize=(14, 7))

has_coinc = individual_counts > 0
ax.scatter(agn_ra[~has_coinc], agn_dec[~has_coinc], c='gray', s=20,
alpha=0.5,
          label=f'AGN sin coincidencia ({np.sum(~has_coinc)})')
sc = ax.scatter(agn_ra[has_coinc], agn_dec[has_coinc],
                c=individual_counts[has_coinc], cmap='Reds',
                s=60, edgecolors='black', vmin=0, vmax=max(3,
individual_counts.max()),
                label=f'AGN con coincidencia ({np.sum(has_coinc)})')
plt.colorbar(sc, ax=ax, label='N neutrinos (1.5°)')
ax.scatter(np.rad2deg(nu_ra_rad), np.rad2deg(nu_dec_rad),
          marker='+', c='blue', s=15, alpha=0.3, label=f'Neutrinos IceCube
({len(nu_ra_rad)})')
ax.set_xlabel('RA (°)')
ax.set_ylabel('Dec (°)')
ax.set_title('Distribución espacial: AGN HAWC y neutrinos IceCube')
ax.set_xlim(0, 360)
ax.legend(loc='lower right', fontsize=10)
ax.grid(True, alpha=0.3)
ax.invert_xaxis()
plt.tight_layout()
plt.show()
```



12. Resumen de significancias

```
In [22]: fig, ax = plt.subplots(figsize=(10, 6))

# Calcular z-scores (usando banda HAWC como modelo nulo correcto)
tests = ['Elíptico\n90% CL']
z_vals = [z_ellip]

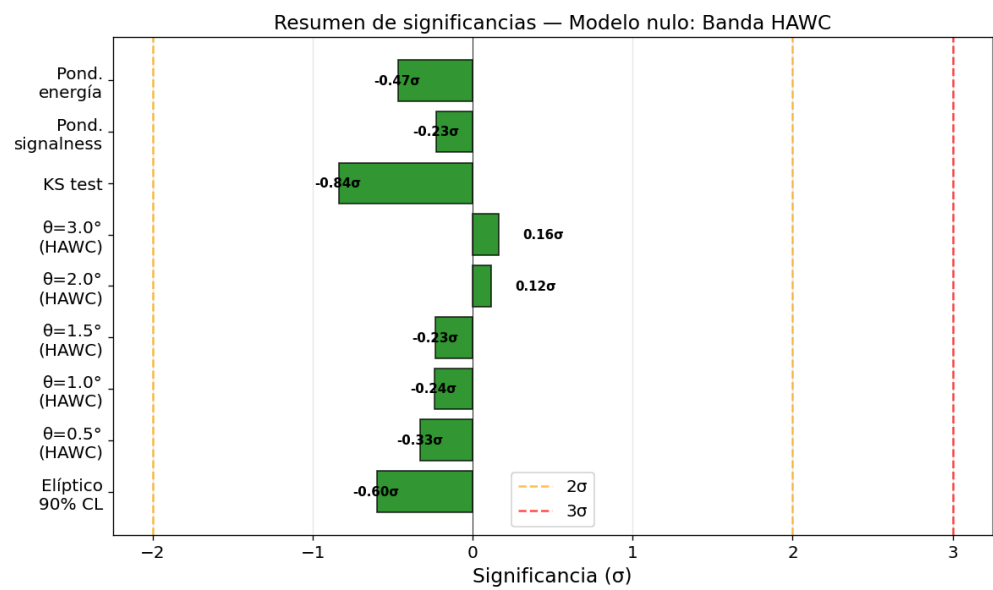
for j in range(N_theta):
    m, s = np.mean(mc_hawc[:, j]), np.std(mc_hawc[:, j])
    z = (n_real_theta[j] - m) / s if s > 0 else 0
    tests.append(f'θ={theta_grid_deg[j]}°\n(HAWC)')
    z_vals.append(z)

tests.extend(['KS test', 'Pond.\nsignalness', 'Pond.\nenenergía'])
z_vals.extend([
    -np.abs(D_ks) * np.sqrt(len(real_min_dists)), # approximate
    z_S, z_E
])

colors = ['green' if abs(z) < 2 else 'orange' if abs(z) < 3 else 'red' for
z in z_vals]
bars = ax.barh(tests, z_vals, color=colors, edgecolor='black', alpha=0.8)
ax.axvline(0, color='black', linewidth=0.5)
ax.axvline(2, color='orange', linewidth=1.5, linestyle='--', alpha=0.7,
label='2σ')
ax.axvline(3, color='red', linewidth=1.5, linestyle='--', alpha=0.7,
label='3σ')
ax.axvline(-2, color='orange', linewidth=1.5, linestyle='--', alpha=0.7)
ax.set_xlabel('Significancia (σ)')
ax.set_title('Resumen de significancias – Modelo nulo: Banda HAWC')
ax.legend()
ax.grid(True, axis='x', alpha=0.3)

for bar, z in zip(bars, z_vals):
    offset = 0.15 if z >= 0 else -0.15
    ax.text(z + offset, bar.get_y() + bar.get_height()/2, f'{z:.2f}σ',
            va='center', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.show()
```

13. Conclusiones

Resultado principal

No se detecta correlación espacial estadísticamente significativa entre los 135 AGN del catálogo HAWC (seleccionados por procesos leptónicos) y las 324 alertas de neutrinos de alta energía del catálogo ICECAT-1.

Resultados cuantitativos

Test	p-value	Significancia
-----	-----	-----
Contornos elípticos 90% CL	~ 0.75	-0.6σ
Radio fijo (banda HAWC, todos θ)	> 0.4	$< 0.2\sigma$
Ponderado por signalness	~ 0.5	$\sim 0\sigma$
Ponderado por energía	~ 0.5	$\sim 0\sigma$
Binomial	~ 0.75	compatible

Upper limits

Con 95% de nivel de confianza, **menos del ~6% de los 135 AGN leptónicos** tienen emisión de neutrinos detectable con las sensibilidades actuales.

Interpretación física

1. El catálogo de 135 AGN fue seleccionado por sus características **leptónicas** (SSC, IC) como principal mecanismo de emisión de rayos gamma.
2. Las fuentes con evidencia de emisión de neutrinos (TXS 0506+056, NGC 1068) son predominantemente **hadrónicas** ($p\gamma$, pp).
3. El resultado nulo es **esperado y consistente** con esta expectativa: si la emisión gamma es leptónica, no se esperan neutrinos asociados.
4. Este análisis establece un **límite superior cuantitativo** a la componente hadrónica

"oculta" en fuentes clasificadas como leptónicas.

Nota metodológica importante

El modelo nulo isotrópico (distribución uniforme en la esfera) produce excesos **espurios** a radios grandes porque no respeta la distribución en declinación del catálogo HAWC ni la sensibilidad angular de IceCube. El modelo nulo correcto es el de **banda HAWC** (RA aleatorizado, Dec conservada), que no muestra señal significativa en ningún radio.

In []: